

Terraform Interview Preparation

Structured Q&A Guide for DevOps / Cloud / SRE Roles

Covers: Fundamentals · Architecture · State Management · Remote Backends · Modules · Variables ·
Meta-Arguments · AWS Provisioning · CI/CD · Security · Troubleshooting · Best Practices · Scenario
Questions

1. Terraform Fundamentals

Q: What is Terraform?

Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp. It lets you define, provision, and manage cloud infrastructure using a declarative configuration language called HCL (HashiCorp Configuration Language). You describe what infrastructure you want, and Terraform figures out how to create it.

Q: Why is Terraform used? What problems does it solve?

- Manual infrastructure setup is error-prone and hard to repeat — Terraform automates it.
- Teams can version-control infrastructure just like application code.
- It provides a consistent, repeatable way to create identical environments (dev, staging, prod).
- It tracks infrastructure state so it knows what already exists and what needs to change.
- Supports multiple cloud providers (AWS, GCP, Azure) from a single tool.

Q: What is Infrastructure as Code (IaC)?

IaC means managing infrastructure (servers, networks, databases) through code files instead of manual clicks in a console. These files can be stored in Git, reviewed, tested, and reused — just like application code.

Q: How does Terraform differ from CloudFormation?

Feature	Terraform	CloudFormation
Provider	HashiCorp (open-source)	AWS (proprietary)
Cloud Support	Multi-cloud (AWS, GCP, Azure...)	AWS only
Language	HCL	JSON / YAML
State Management	Manages its own state file	AWS manages state internally
Modules	Strong, reusable module system	Nested stacks (less flexible)

Q: How does Terraform differ from Ansible?

Aspect	Terraform	Ansible
Primary Use	Provision infrastructure	Configure software on servers
Approach	Declarative	Procedural (playbooks)
State	Maintains state file	No built-in state
Best For	Creating VPCs, EKS, RDS	Installing packages, deploying apps

Note: In practice, Terraform and Ansible are often used together: Terraform provisions the infrastructure, Ansible configures it.

Q: What are Terraform providers, resources, and data sources?

Provider: A plugin that lets Terraform talk to a specific platform (e.g., AWS, Azure). You declare it at the top of your config. Example: provider 'aws' sets the region and credentials.

Resource: An actual infrastructure object you want to create or manage — like an EC2 instance, S3 bucket, or VPC. Defined using a resource block.

Data Source: Used to read existing infrastructure that Terraform did not create. For example, fetching an existing AMI ID or VPC ID to reference in your code.

Q: Explain the Terraform workflow.

- **terraform init** — Downloads providers and modules. Run first.
- **terraform plan** — Shows what will be created, changed, or destroyed. No actual changes.
- **terraform apply** — Executes the plan. Creates/modifies/destroys real infrastructure.
- **terraform destroy** — Tears down all resources managed by the configuration.

2. Terraform Lifecycle & Commands

Q: terraform init

Initializes working directory. Downloads provider plugins and modules. Must run first.

Q: terraform validate

Checks if config files are syntactically valid. Does not talk to AWS.

Q: terraform fmt

Formats .tf files to the standard style. Good for code consistency.

Q: terraform plan

Creates an execution plan. Shows what will change before actually changing it.

Q: terraform apply

Applies the plan. Provisions or modifies real infrastructure.

Q: terraform destroy

Destroys all resources defined in the config.

Q: terraform output

Displays output values defined in your config (e.g., VPC ID, EKS endpoint).

Q: terraform state list

Lists all resources tracked in the state file.

Q: terraform import

Imports existing infrastructure into Terraform state without recreating it.

Q: terraform refresh

Syncs the state file with the real infrastructure. Detects drift.

Q: terraform show

Displays the current state or a saved plan in human-readable form.

Q: terraform graph

Generates a visual dependency graph of resources.

Q: What happens internally during terraform apply?

- Reads the current state file to understand existing infrastructure.
- Calls provider APIs (e.g., AWS) to refresh actual resource states.
- Computes a diff between desired state (your .tf files) and current state.
- Builds a dependency graph to determine creation order.
- Executes API calls in parallel where possible (no dependencies).
- Updates the state file after each resource is created/modified/destroyed.

Q: How does Terraform handle parallel execution?

Terraform builds a resource dependency graph (DAG). Resources with no dependencies are provisioned in parallel. Resources that depend on another (via references or `depends_on`) wait until their dependency is complete. Default parallelism is 10 concurrent operations.

3. Terraform State Management

Q: What is Terraform state and why is it required?

Terraform stores a mapping between the resources defined in your .tf files and the real infrastructure objects in the cloud. This is saved in a file called terraform.tfstate. Without it, Terraform would have no way to know what already exists, what to update, or what to delete.

Q: What information is stored in state?

- Resource IDs (e.g., EC2 instance ID, VPC ID)
- Metadata and attributes of each resource
- Dependency relationships between resources
- Sensitive values like passwords (this is why state must be secured)

Q: What are the risks of local state?

- No sharing — only one person can work on infrastructure at a time.
- No locking — two people running apply simultaneously can corrupt state.
- No backup — if the file is deleted or lost, Terraform loses track of all resources.
- Security risk — state may contain sensitive values stored in plain text.

Q: What is remote state?

Storing the state file in a shared, centralized location (e.g., AWS S3) instead of locally. This allows teams to collaborate, enables state locking, and provides backup and versioning.

Q: What is state locking and why is it important?

State locking prevents two Terraform processes from modifying the state at the same time. Without locking, simultaneous applies from two engineers can corrupt the state file. When using S3 + DynamoDB backend, DynamoDB provides the lock — Terraform writes a lock entry before apply and removes it after.

Q: What is state drift? How do you detect and handle it?

Drift means the actual infrastructure has changed outside Terraform (e.g., someone manually deleted or modified a resource in the AWS Console).

- **Detect:** Run 'terraform plan' — it will show differences between state and real infrastructure.
- **Refresh:** Run 'terraform refresh' to update the state file to match reality.
- **Reconcile:** Run 'terraform apply' to bring real infrastructure back to desired state.
- **Prevent:** Use IAM policies to restrict manual changes to Terraform-managed resources.

Q: What is terraform state rm / mv / import?

terraform state rm: Removes a resource from state without destroying the actual resource. Useful when you want Terraform to stop managing something.

terraform state mv: Moves a resource to a different address in state. Used when refactoring code (e.g., moving a resource into a module).

terraform import: Imports an existing real resource into Terraform state so it can be managed going forward, without recreating it.

Q: How do you recover a corrupted state file?

- Restore from S3 versioning — S3 keeps previous versions of the state file.
- Use 'terraform state list' and 'terraform show' to inspect the current state.
- Manually rebuild state using 'terraform import' for each affected resource.
- Use 'terraform refresh' to re-sync state with real infrastructure.

Note: This is why S3 versioning must always be enabled for Terraform state buckets.

4. Remote Backend (S3 + DynamoDB)

Q: How do you configure an S3 backend with DynamoDB locking?

```
terraform {  
  backend "s3" {  
    bucket = "my-terraform-state"  
    key = "prod/terraform.tfstate"  
    region = "us-east-1"  
    dynamodb_table = "terraform-lock"  
    encrypt = true  
  }  
}
```

Q: Why use DynamoDB with S3 for Terraform state?

- S3 stores the state file — it provides durability, versioning, and encryption.
- DynamoDB provides state locking — it prevents two engineers from running apply at the same time.
- DynamoDB uses a LockID attribute. Terraform creates a lock entry before apply and deletes it after.
- If an apply is interrupted, the lock may remain. You can force-unlock with: `terraform force-unlock`

Q: What happens if two engineers run terraform apply simultaneously?

With DynamoDB locking enabled: the second engineer gets an error saying the state is locked and apply is refused. Without locking: both processes modify the state file simultaneously, causing corruption and unpredictable infrastructure changes.

Q: How do you secure Terraform state?

- Enable S3 server-side encryption (SSE-S3 or SSE-KMS).
- Enable S3 bucket versioning to recover previous state versions.
- Block all public access on the S3 bucket.
- Use IAM policies to restrict who can read/write the state bucket.
- Use DynamoDB for locking to prevent concurrent modifications.
- Never commit `.tfstate` files to Git.

5. Variables, Outputs, and Locals

Q: What are input variables and why use them?

Input variables let you parameterize your Terraform code so the same configuration can be reused across environments (dev, staging, prod) without changing the code itself. You define them with a variable block and pass values via .tfvars files or CLI flags.

Q: What is a tfvars file? How do you pass variables?

A .tfvars file contains variable values. Terraform automatically loads terraform.tfvars. For environment-specific files (e.g., prod.tfvars), you pass them explicitly:

```
terraform apply -var-file=prod.tfvars
```

You can also pass individual variables: terraform apply -var='region=us-east-1'

Q: What are output values? How do they help?

Outputs expose specific values after apply — like a VPC ID or EKS cluster endpoint. They are useful for:

- Passing values between Terraform modules (child module outputs used in root module).
- Displaying important resource info after apply.
- Using infrastructure outputs in CI/CD pipelines.

Q: What is the difference between locals and variables?

Aspect	Input Variables	Local Values
Set by	User (via CLI, .tfvars, env vars)	Defined in code, not overridable
Purpose	Accept external input	Compute reusable expressions internally
Example	var.environment = 'prod'	locals { name = 'prod-vpc' }

Q: What is variable validation?

You can add a validation block inside a variable to enforce constraints:

```
variable "environment" {
  type = string
  validation {
    condition = contains(["dev","staging","prod"], var.environment)
    error_message = "Must be dev, staging, or prod."
  }
}
```

6. Terraform Modules

Q: What is a module and why use it?

A module is a reusable package of Terraform configuration files. Instead of repeating the same VPC or EKS code in every environment, you write it once as a module and call it multiple times with different inputs. It improves reusability, consistency, and maintainability.

Q: Difference between root module and child module?

Root module: The main working directory where you run terraform init/apply. It calls other modules.

Child module: A reusable module called by the root module using a module block. It lives in a separate directory or registry.

Q: How do you structure a reusable module?

```
modules/
  vpc/
    main.tf      # Resources
    variables.tf # Input variables
    outputs.tf   # Exported values
  eks/
    main.tf
    variables.tf
    outputs.tf
envs/
  prod/
    main.tf      # Calls modules
    terraform.tfvars
```

Q: How do modules accept inputs and return outputs?

Modules use variable blocks for inputs and output blocks for outputs. The root module passes values in the module call:

```
module "vpc" {
  source = "../modules/vpc"
  cidr_block = "10.0.0.0/16"
  environment = var.environment
}
# Accessing output from the module:
resource "aws_eks_cluster" "main" {
  vpc_config { subnet_ids = module.vpc.private_subnet_ids }
}
```

Q: How do you version modules?

- Use Git tags for versioning: source = 'git::https://github.com/org/modules.git?ref=v1.2.0'
- Use Terraform Registry with semantic versioning: source = 'terraform-aws-modules/eks/aws', version = '~> 19.0'
- Pin versions in production to avoid unexpected changes from upstream module updates.

7. Meta-Arguments

Q: What is count? What is for_each? What is the difference?

count: Creates N identical copies of a resource using an integer. Good for simple repetition.

```
resource "aws_instance" "web" { count = 3 } # creates 3 instances
```

for_each: Creates one resource per item in a map or set. Each resource gets a unique key and can have different attributes.

```
resource "aws_s3_bucket" "envs" {  
  for_each = toset(["dev", "staging", "prod"])  
  bucket = "myapp-${each.key}"  
}
```

Key difference: count is index-based (0, 1, 2) — deleting middle item shifts indices and Terraform recreates resources. for_each is key-based — removing one item only affects that one resource. Prefer for_each in production.

Q: What is depends_on and when do you use it?

depends_on explicitly tells Terraform that one resource must be created before another, even when there is no direct reference between them. Use it when Terraform cannot automatically detect the dependency.

```
resource "aws_eks_cluster" "main" {  
  depends_on = [aws_iam_role_policy_attachment.eks_policy]  
}
```

Q: What is the lifecycle block? Explain create_before_destroy, prevent_destroy, and ignore_changes.

create_before_destroy: Creates the replacement resource before destroying the old one. Avoids downtime during updates.

prevent_destroy: Prevents Terraform from destroying a resource. Useful for production databases. Apply will fail if destruction is attempted.

ignore_changes: Tells Terraform to ignore changes to specific attributes. Useful when a resource is modified outside Terraform (e.g., auto-scaling changes instance count).

```
lifecycle {  
  create_before_destroy = true  
  prevent_destroy = true  
  ignore_changes = [tags, desired_capacity]  
}
```

8. AWS Provisioning with Terraform

Q: How do you create a VPC with public and private subnets?

- Define aws_vpc resource with CIDR block.
- Create aws_subnet resources — public subnets with map_public_ip_on_launch = true.
- Create aws_internet_gateway and attach it to the VPC.
- Create aws_route_table for public subnets with route to internet gateway.
- Create aws_eip and aws_nat_gateway in a public subnet for private subnet outbound traffic.
- Create separate aws_route_table for private subnets with route to NAT gateway.
- Associate route tables to subnets using aws_route_table_association.

Q: How did you provision an EKS cluster using Terraform?

- Used the terraform-aws-modules/eks/aws community module (battle-tested, widely used).

- Defined VPC, subnets, and security groups first.
- Created IAM roles: one for the EKS control plane, one for worker node groups.
- Configured node groups with instance types, scaling policies, and desired capacity.
- Set up OIDC provider for IRSA (IAM Roles for Service Accounts).
- Configured aws_auth ConfigMap to grant kubectl access to IAM users/roles.

Q: How do you manage RDS with Terraform?

- Use aws_db_instance or aws_rds_cluster for Aurora.
- Store the master password in AWS Secrets Manager — never hardcode it in .tf files.
- Enable multi_az = true for production high availability.
- Set backup_retention_period to at least 7 days.
- Place RDS in private subnets using aws_db_subnet_group.
- Use aws_security_group to allow access only from application security groups.

Q: How do you configure IAM roles in Terraform?

```
resource "aws_iam_role" "eks_node" {
  name = "eks-node-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}
```

9. Terraform in CI/CD (Jenkins & GitHub Actions)

Q: How do you integrate Terraform into a Jenkins pipeline?

- Install Terraform binary on Jenkins agent or use a Docker image with Terraform.
- Store AWS credentials as Jenkins Credentials (not in Jenkinsfile).
- Use withCredentials block to inject AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY.
- Pipeline stages: Checkout → terraform init → terraform validate → terraform plan → Manual Approval → terraform apply.
- For production, require a manual approval step before apply.
- Archive plan output as an artifact for audit trail.

Q: How do you integrate Terraform into GitHub Actions?

```
jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: hashicorp/setup-terraform@v2
      - run: terraform init
      - run: terraform plan
      env:
        AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
        AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
      - run: terraform apply -auto-approve # Only on main branch
```

Note: Use GitHub Environments with required reviewers to gate production applies.

Q: How do you manage secrets in Terraform pipelines?

- AWS credentials: Use IAM roles for EC2/Jenkins agents instead of static keys.
- For GitHub Actions: Use OpenID Connect (OIDC) to assume an IAM role — no long-lived keys needed.
- Sensitive variable values: Store in AWS Secrets Manager or HashiCorp Vault.
- In pipelines: Use environment secrets (GitHub Secrets, Jenkins Credentials).
- Never put secrets in .tf files, .tfvars files committed to Git, or pipeline logs.

Q: How do you handle rollbacks in Terraform?

Terraform does not have a built-in rollback command. Rollback strategies:

- Revert the Git commit that introduced the bad change and re-run apply.
- Restore previous state from S3 versioning and re-run apply.
- For destructive changes: use prevent_destroy lifecycle rule to prevent accidental deletes.
- Use plan output review process — always review plan output before approving apply.

10. Terraform Security

Q: How do you avoid hardcoding secrets in Terraform?

- Use variable blocks — pass secrets via environment variables (TF_VAR_db_password) or CI/CD secrets.
- Mark sensitive variables with sensitive = true to suppress output in logs.
- Use AWS Secrets Manager data source to fetch secrets at runtime.
- Use IRSA (IAM Roles for Service Accounts) for EKS workloads — no static credentials needed.

Q: What is IAM least privilege in the context of Terraform?

Terraform only needs the exact IAM permissions required to create/manage the resources in your config — not AdministratorAccess. Create a dedicated Terraform IAM role with scoped permissions. For example, if Terraform only manages VPCs, it should only have `ec2:*` permissions on VPC-related actions, not S3 or RDS permissions.

Q: How do you scan Terraform code for security issues?

- **Checkov:** Static analysis tool that checks for misconfigurations (e.g., S3 bucket is public, security group allows 0.0.0.0/0).
- **tfsec:** Terraform-specific security scanner. Checks for common AWS security issues.
- **Trivy:** Can scan Terraform configs in addition to container images and filesystems.
- Integrate these in CI/CD pipelines to fail builds on security violations.

11. Terraform Troubleshooting

Q: terraform apply failed — what do you do?

- Read the error message carefully — Terraform usually tells you exactly which resource failed.
- Check if the failure is an API permission issue (add IAM permissions), resource limit, or naming conflict.
- Run terraform plan again after fixing the issue to verify the fix before re-applying.
- Partially created resources remain in state — Terraform will handle them on the next apply.

Q: State lock exists — how do you resolve it?

- This happens when a previous apply was interrupted and the DynamoDB lock entry was not removed.
- First, verify no other apply is actually running.
- Run: terraform force-unlock to manually release the lock.
- Get the LOCK_ID from the error message or from the DynamoDB table.

Q: Resource already exists in AWS but not in state — how do you fix it?

- Use terraform import to bring the existing resource into Terraform state.
- Example: terraform import aws_vpc.main vpc-0abc123
- After import, run terraform plan to verify the config matches the existing resource.
- Adjust your .tf code if plan shows unwanted changes.

Q: Infrastructure drift detected — what do you do?

- Run terraform plan to see what changed outside Terraform.
- Decision: Either re-apply to restore desired state, or update .tf code to match reality.
- For accidental manual changes: apply to restore Terraform-managed config.
- Prevent future drift: restrict manual access via IAM policies.

Q: Someone manually deletes an EC2 instance created by Terraform — what happens next plan?

- terraform plan shows: 'aws_instance.web must be created' (1 to add).
- Terraform detects the resource is missing from real infrastructure.
- Running terraform apply recreates the instance from scratch.
- This is normal and expected Terraform behavior.

Q: Apply hangs indefinitely — what do you do?

- Check AWS Console — the resource may be in a pending/transitioning state.
- Check for circular dependencies in your resource graph.
- Check IAM permissions — some actions fail silently and cause hangs.
- Set TF_LOG=DEBUG for verbose logging to identify where it is stuck.

12. Terraform Best Practices

Q: Always use remote state

Never use local state for team projects. Store state in S3 + DynamoDB.

Q: Use modules for reusability

Never copy-paste the same VPC/EKS code across environments. Write once, reuse.

Q: Separate environments

Use separate state files per environment (dev/staging/prod). Prevents accidental prod changes.

Q: Never hardcode values

Use variables and .tfvars files. Never hardcode account IDs, AMI IDs, or passwords.

Q: Pin provider versions

Always pin provider versions in required_providers to avoid breaking changes.

Q: Review plan before apply

Always review 'terraform plan' output in CI/CD. Add manual approval gates for prod.

Q: Use prevent_destroy

Add lifecycle prevent_destroy = true for production databases and critical resources.

Q: Run terraform fmt

Enforce consistent formatting in CI/CD. Fail pipeline if files are not formatted.

Q: Scan for security issues

Run Checkov or tfsec in CI/CD pipeline before apply.

Q: Enable S3 versioning

Always enable versioning on the Terraform state bucket for recovery.

Q: Use for_each over count

Prefer for_each for resources — avoids index-shift issues when adding/removing items.

Q: Tag all resources

Define a common tags local variable and apply it to every resource for cost tracking.

13. Real-World Scenario Questions

Q: Scenario: Terraform code accidentally destroys production resources. How do you prevent this?

- Add lifecycle { prevent_destroy = true } to all critical resources (RDS, EKS cluster).
- Use separate state files per environment — accidental apply to prod is harder.
- Require manual approval step in CI/CD before apply on prod.
- Use Terraform workspaces or separate backends to isolate environments.
- Run policy checks with Sentinel (Terraform Enterprise) or Open Policy Agent to block destructive plans.
- Enable S3 versioning on state bucket — you can restore previous state after an accident.

Q: Scenario: EKS deployment fails because VPC was not created first. How do you solve it?

- This usually means the EKS module is referencing VPC outputs that don't exist yet.
- The fix: ensure module 'eks' explicitly depends on module 'vpc'.
- Either reference VPC outputs (automatic dependency): subnet_ids = module.vpc.private_subnet_ids
- Or use depends_on = [module.vpc] inside the EKS module call if there's no direct reference.
- Terraform's dependency graph handles ordering automatically when references are used correctly.

Q: Scenario: State file becomes corrupted. What are your recovery options?

- Restore previous state version from S3 versioning (most reliable option).
- Download the previous version from S3: `aws s3 cp s3://bucket/prod/terraform.tfstate?versionId=... terraform.tfstate`
- Use terraform import to re-import resources that cannot be recovered from state.
- Run terraform refresh to sync any partially recovered state with real infrastructure.
- Lesson learned: always enable S3 versioning and test recovery procedures.

Q: Scenario: Terraform deployment succeeds but application access fails. How do you troubleshoot?

- Check security groups — confirm the correct ports are open between load balancer, app, and RDS.
- Check route tables — ensure private subnets have a route to NAT gateway for outbound traffic.
- Check IAM roles — verify the application pod/instance has the correct IAM permissions.
- Check network ACLs — they can block traffic even if security groups are correct.
- Check DNS resolution — confirm Route53 or load balancer DNS is pointing to the correct target.
- Check EKS aws-auth ConfigMap — verify application service accounts have the correct IAM role binding.

14. Experience-Based & Behavioral Questions

Q: How would you explain your Terraform project architecture?

Structure your answer using this framework:

- **What:** Briefly describe the project (e.g., 'I provisioned a full AWS EKS-based GitOps platform using Terraform').
- **Structure:** Describe your module layout — separate modules for VPC, EKS, IAM, RDS.
- **State:** Explain your S3 + DynamoDB backend setup and why you chose it.
- **CI/CD:** Describe how Jenkins or GitHub Actions ran Terraform (plan on PR, apply on merge to main).
- **Challenges:** Mention one real issue you solved (e.g., state lock, drift, dependency ordering).

Q: What improvements would you make to your Terraform code today?

- Add Checkov/tfsec scanning to the CI/CD pipeline for automated security checks.
- Add variable validation blocks to catch invalid inputs early.
- Move to OIDC-based AWS authentication in GitHub Actions to eliminate long-lived credentials.
- Add automated drift detection using a scheduled terraform plan in CI/CD.
- Improve tagging strategy for better cost allocation across environments.

Q: Tell me about a Terraform deployment failure you experienced.

Answer structure (STAR format):

- **Situation:** Describe the context (e.g., applying EKS config in staging).
- **Task:** What you were trying to achieve.
- **Action:** Specific steps you took to debug (checking logs, state, AWS console).
- **Result:** How you resolved it and what you learned.
- **Tip:** Mention a real issue if possible — like a state lock, missing IAM permission, or dependency ordering problem.

Q: How do you collaborate with developers on IaC?

- All Terraform code lives in Git — developers can raise PRs for infrastructure changes.
- terraform plan output is posted as a PR comment (via GitHub Actions) for review before merge.
- Use module interfaces — developers don't need to know Terraform internals, just the module inputs.
- Enforce terraform fmt and validate in CI to maintain code quality.
- Use pull request reviews for infrastructure changes just like application code reviews.

Quick Reference: High-Priority Topics

Topic	Why It Matters
Terraform workflow (init/plan/apply/destroy)	Asked in every interview — know each step deeply
State files and remote state	Core concept — understand what's stored, why it matters
S3 + DynamoDB backend	Standard AWS setup — be ready to explain your config
Variables, outputs, locals	Used in every real project — know the differences
Modules and reusability	Key differentiator — explain a real module you built
count vs for_each	Frequently asked — know when to use which and why
depends_on and lifecycle	Shows deep Terraform knowledge
VPC provisioning	Core AWS networking — subnets, IGW, NAT, route tables
EKS provisioning	Your featured project — explain every resource
IAM roles and policies	Security fundamentals — know least privilege and IRSA
Jenkins/GitHub Actions + Terraform	Your CI/CD experience — plan→approve→apply flow
Troubleshooting state lock, drift, import	Real-world scenarios — show problem-solving ability
Security (no hardcoded secrets, scanning)	Shows maturity beyond basics

Tip: For every answer, try to connect it to your actual internship project at Hisan Labs. Real examples from your EKS GitOps Pipeline or Serverless Game Hosting Platform will make your answers significantly stronger than textbook definitions.